WEEK-09

Implement All Pair Shortest paths problem using Floyd's algorithm.

Start here | flyod.c

```c
#include <stdio.h>
#include <stdlib.h>

#define INF 99999

void floydWarshall(int **matrix, int v) {
    for (int k = 0; k < v; k++) {
        for (int i = 0; i < v; i++) {
            for (int j = 0; j < v; j++) {
                if (matrix[i][k] != INF && matrix[k][j] != INF &&
                    matrix[i][k] + matrix[k][j] < matrix[i][j]) {
                    matrix[i][j] = matrix[i][k] + matrix[k][j];
                }
            }
        }
    }
}

int main() {
    int v, e;

    printf("Enter the number of vertices: ");

    scanf("%d", &v);
    printf("Enter the number of edges: ");
    scanf("%d", &e);

    // Memory allocation
    int **matrix = (int **)malloc(v * sizeof(int *));
    for (int i = 0; i < v; i++) {
        matrix[i] = (int *)malloc(v * sizeof(int));
        for (int j = 0; j < v; j++) {
            if (i == j) matrix[i][j] = 0;
            else matrix[i][j] = INF;
        }
    }

    printf("\nEnter edges in the format: [Source] [Destination] [Weight]\n");
    printf("(Note: Vertex indices must range from 0 to %d)\n", v - 1);

    for (int i = 0; i < e; i++) {
        int u, src, dest, weight;
        printf("Edge %d: ", i + 1);
        scanf("%d %d %d", &src, &dest, &weight);

```

```
46          if (src >= 0 && src < v && dest >= 0 && dest < v) {
47              matrix[src][dest] = weight;
48          } else {
49              printf("Invalid vertex indices! Try again.\n");
50              i--; // Repeat this iteration
51          }
52      }
53
54      floydWarshall(matrix, v);
55
56      printf("\nThe final shortest distance matrix between every pair of vertices:\n");
57      for (int i = 0; i < v; i++) {
58          for (int j = 0; j < v; j++) {
59              if (matrix[i][j] >= INF)
60                  printf("%7s", "INF");
61              else
62                  printf("%7d", matrix[i][j]);
63          }
64          printf("\n");
65      }
66
67      // Free memory
68      for (int i = 0; i < v; i++) free(matrix[i]);
69      free(matrix);
70
71      return 0;
72  }
73
```

```
"C:\Users\BMSCECSE\Docume
Enter the number of vertices: 4
Enter the number of edges: 4

Enter edges in the format: [Source] [Destination] [Weight]
(Note: Vertex indices must range from 0 to 3)
Edge 1: 0 1 5
Edge 2: 0 3 10
Edge 3: 1 2 3
Edge 4: 2 3 1

The final shortest distance matrix between every pair of vertices:
      0      5      8      9
    INF      0      3      4
    INF    INF      0      1
    INF    INF    INF      0

Process returned 0 (0x0)   execution time : 30.164 s
Press any key to continue.
```

LEETCODE: 1334. Find the City With the Smallest Number of Neighbors at a Threshold Distance

1334. Find the City With the Smallest Number of Neighbors at a Threshold Distance Solved

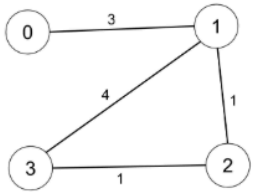
Medium Topics Companies Hint

There are `n` cities numbered from `0` to `n-1`. Given the array `edges` where `edges[i] = [from_i, to_i, weight_i]` represents a bidirectional and weighted edge between cities `from_i` and `to_i`, and given the integer `distanceThreshold`.

Return the city with the smallest number of cities that are reachable through some path and whose distance is **at most** `distanceThreshold`, If there are multiple such cities, return the city with the greatest number.

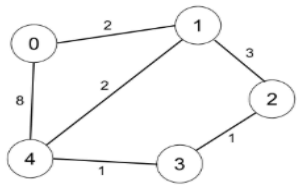
Notice that the distance of a path connecting cities ***i*** and ***j*** is equal to the sum of the edges' weights along that path.

Example 1:




```
Input: n = 4, edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]], distanceThreshold = 4
Output: 3
Explanation: The figure above describes the graph.
The neighboring cities at a distanceThreshold = 4 for each city are:
City 0 -> [City 1, City 2]
City 1 -> [City 0, City 2, City 3]
City 2 -> [City 0, City 1, City 3]
City 3 -> [City 1, City 2]
Cities 0 and 3 have 2 neighboring cities at a distanceThreshold = 4, but we have to return city 3 since it has the
greatest number.
```

Example 2:




```
Input: n = 5, edges = [[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]], distanceThreshold = 2
Output: 0
Explanation: The figure above describes the graph.
The neighboring cities at a distanceThreshold = 2 for each city are:
City 0 -> [City 1]
City 1 -> [City 0, City 4]
City 2 -> [City 3, City 4]
City 3 -> [City 2, City 4]
City 4 -> [City 1, City 2, City 3]
The city 0 has 1 neighboring city at a distanceThreshold = 2.
```

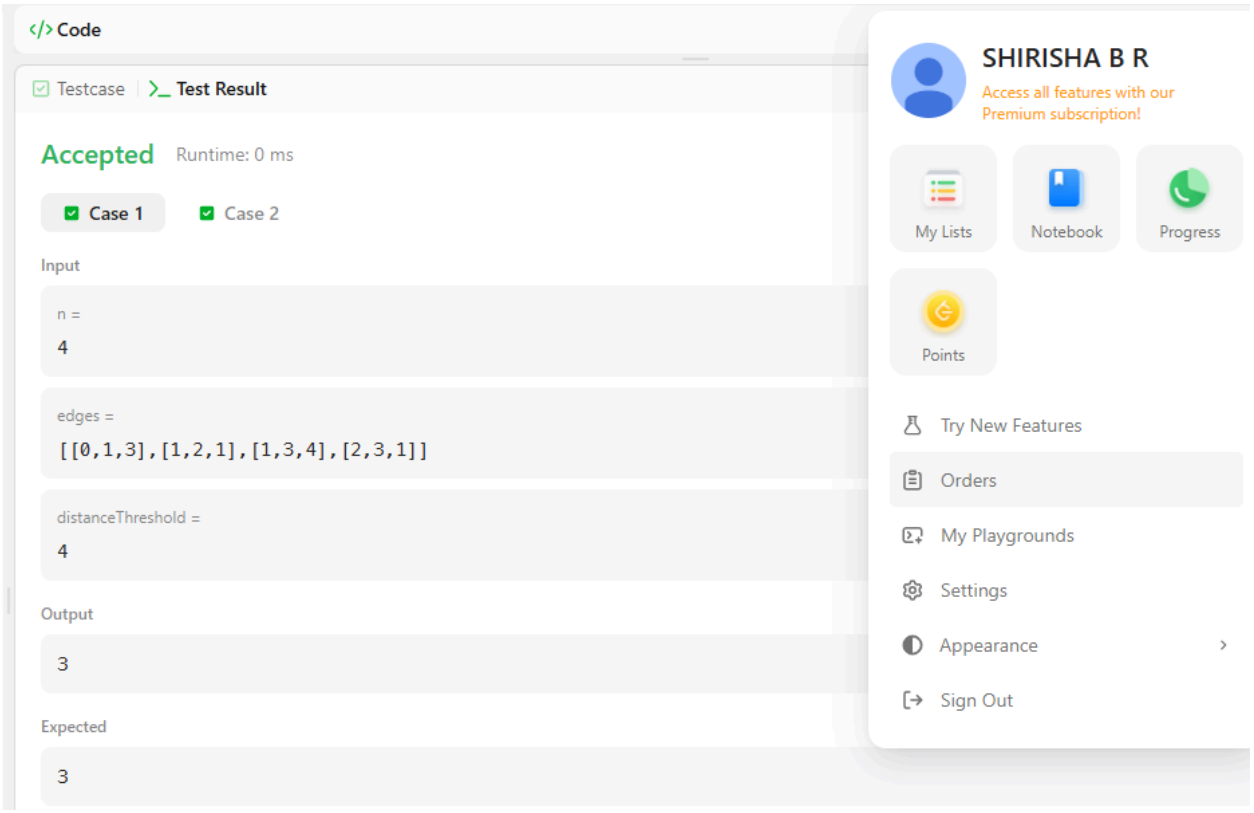
Constraints:

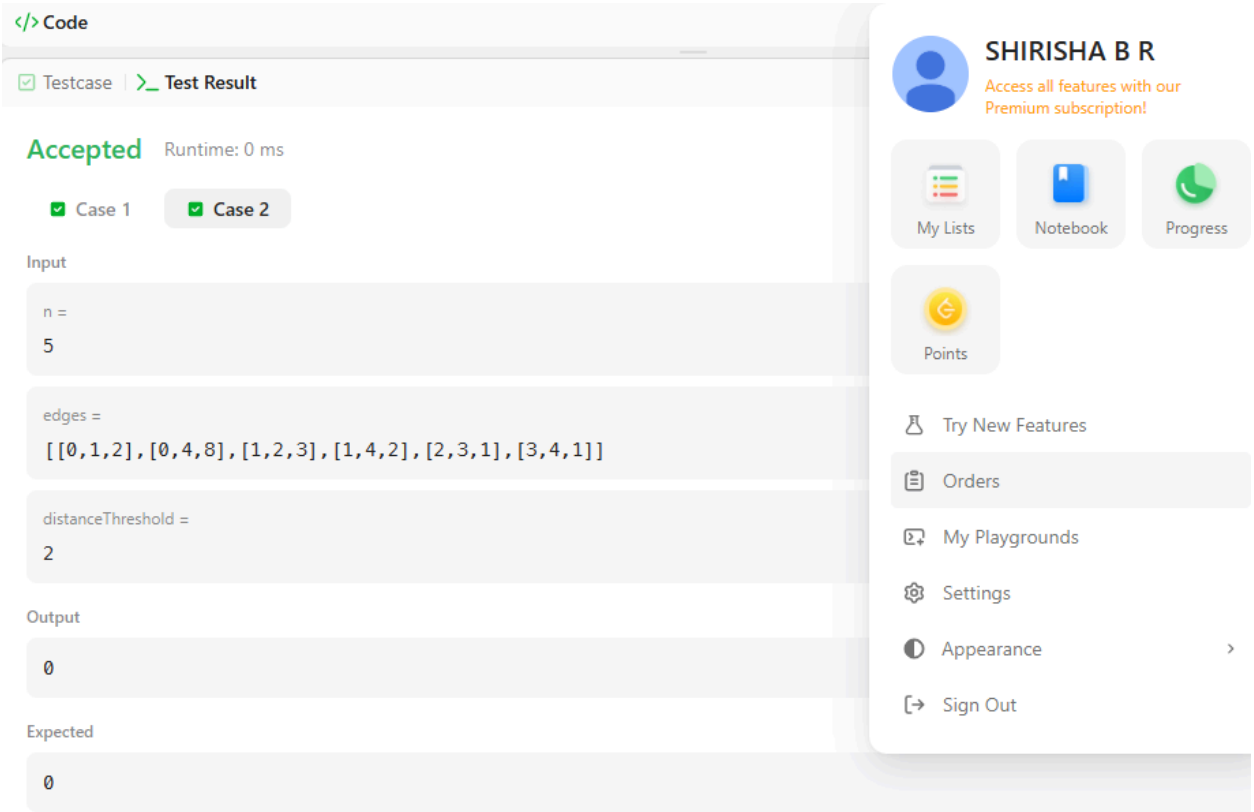
- `2 <= n <= 100`
- `1 <= edges.length <= n * (n - 1) / 2`
- `edges[i].length == 3`
- `0 <= from_i < to_i < n`
- `1 <= weight_i, distanceThreshold <= 10^4`
- All pairs `(from_i, to_i)` are distinct.

</> Code

C ⌄ 🔒 Auto

```c
#define MIN(a, b) ((a) < (b) ? (a) : (b))

int findTheCity(int n, int** edges, int edgesSize, int* colSize, int th) {
    int i, j, k, cnt, d[100][100], min = n, res = 0;

    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) d[i][j] = (i == j) ? 0 : 10001;

    for (i = 0; i < edgesSize; i++)
        d[edges[i][0]][edges[i][1]] = d[edges[i][1]][edges[i][0]] = edges[i][2];

    for (k = 0; k < n; k++)
        for (i = 0; i < n; i++)
            if (d[i][k] <= th)
                for (j = 0; j < n; j++)
                    d[i][j] = MIN(d[i][j], d[i][k] + d[k][j]);

    for (i = 0; i < n; i++) {
        for (cnt = 0, j = 0; j < n; j++) if (d[i][j] <= th) cnt++;
        if (cnt <= min) { min = cnt; res = i; }
    }
    return res;
}
```


Code
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
n =
4
edges =
[[0,1,3],[1,2,1],[1,3,4],[2,3,1]]
distanceThreshold =
4
Output
3
Expected
3
SHIRISHA B R
Access all features with our Premium subscription!
My Lists
Notebook
Progress
Points
Try New Features
Orders
My Playgrounds
Settings
Appearance
Sign Out


Code
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
n =
5
edges =
[[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]]
distanceThreshold =
2
Output
0
Expected
0
SHIRISHA B R
Access all features with our Premium subscription!
My Lists
Notebook
Progress
Points
Try New Features
Orders
My Playgrounds
Settings
Appearance
Sign Out